

INGENIERÍA DE SISTEMAS

Tecnología Web II

ACTIVIDAD ACADÉMICA N°2

Desarrollo del Backend y API REST

VisualSync

AUTORES

David Miguel Flores Medrano

Docente: Vladimir Wilmar Rojas Condori

Cochabamba-Bolivia

18-10-2026

INDICE DE CONTENIDO

Cambio de herramientas respecto a las sugeridas en la guía 1

Consulta realizada al docente. 1

Comparación de herramientas 1

Justificación del cambio 2

1.Estructura del proyecto backend 3

2. Implementación con C# y ASP.NET Core 4

3. Arquitectura del backend 5

Decisiones de seguridad 7

4. Configuración de PostgreSQL 7

5. Modelo y tablas de la base de datos 8

Usuarios 8

PaquetesServicio 8

Reservas 9

ArchivosGaleria 10

6. Conexión mediante ORM 11

7. Migraciones 13

8. Operaciones CRUD 13

Ejemplo de controlador (Paquetes) 14

Reglas de negocio de las reservas 15

9. Validación de datos 15

10. Manejo de errores 16

11. Endpoints REST 18

12. JSON y códigos HTTP 21

Registro: POST /api/auth/registro 21

Respuesta 201 Created: 21

Login: POST /api/auth/login. Respuesta 200 OK: 21

Crear reserva: POST /api/reservas (cabecera Authorization: Bearer token) 21

Respuesta 201 Created: 21

Error de validación (400): 22

Error de negocio (409): 22

Códigos HTTP utilizados 22

Cambio de herramientas respecto a las sugeridas en la guía

La guía de la Actividad Académica N°2 sugiere implementar el backend con Node.js, Express, TypeScript, PostgreSQL y un ORM. En la Actividad 1, la arquitectura de VisualSync fue diseñada con C#, ASP.NET Core y MySQL: los diagramas C4, el modelo de datos, el mapa de rutas y la estructura de carpetas corresponden a ese stack. Antes de comenzar el desarrollo se consultó al docente si era posible continuar con C#.

Consulta realizada al docente.

Buenas tardes, Inge. Tengo una consulta sobre la Actividad 2. En la Actividad 1 estructuré la arquitectura de mi proyecto con C#, ASP.NET Core y MySQL, pero veo que en la nueva guía se sugiere usar Node.js, TypeScript y PostgreSQL. ¿Puedo continuar con el desarrollo en C# que ya diseñé, o es obligatorio que migre el backend al entorno de Node.js para esta evaluación?

Respuesta del docente. El Ing. Vladimir Wilmar Rojas Condori respondió que se puede continuar con el desarrollo en C#, con la condición de que el cambio de herramientas respecto a las sugeridas se mencione en el documento, tal como fue planteado en la consulta. Esta sección cumple esa indicación. La captura de la conversación se incluye en los Anexos.

Decisión adoptada. El proyecto continúa con C#, ASP.NET Core (Web API) y Entity Framework Core. Como base de datos se utiliza PostgreSQL, alojado en Supabase, que es el gestor sugerido por la guía, en lugar del MySQL previsto en la Actividad 1. El modelo de datos, las reglas de negocio, los roles y las historias de usuario no cambian.

Comparación de herramientas

Elemento	**Sugerida en la guía**	**Utilizada en VisualSync**
Lenguaje	JavaScript / TypeScript	C#

Framework	Node.js + Express	.NET + ASP.NET Core Web API
Base de datos	PostgreSQL	PostgreSQL (Supabase)
ORM	ORM de JS/TS	Entity Framework Core + Npgsql
Migraciones	Migraciones del ORM	EF Core Migrations (dotnet ef)
Validación	Librerías de JS	Data Annotations en DTOs
Manejo de errores	Middleware de Express	Middleware global con ProblemDetails
Autenticación	JWT	JWT Bearer + BCrypt
Documentación	Swagger o Postman	Swagger/OpenAPI + Postman
Pruebas	Postman	Postman
Entorno	Visual Studio Code	Visual Studio Code
Control de versiones	Git y GitHub	Git y GitHub

Justificación del cambio

1. Continuidad con la Actividad 1. Los diagramas C4, el modelo de datos, el mapa de rutas y la estructura de carpetas ya estaban definidos para ASP.NET Core y Entity Framework Core. Migrar de plataforma obligaría a rehacerlos y rompería la coherencia entre ambas actividades. El único ajuste de infraestructura es el gestor de base de datos, que pasa de MySQL a PostgreSQL por ser el sugerido en la guía.

2. Equivalencia funcional. Cada punto del contenido mínimo tiene un equivalente directo en .NET: API REST, ORM, migraciones, CRUD, validación, manejo de errores, JSON, códigos HTTP y documentación.
3. Dominio de la plataforma. La experiencia previa con ASP.NET Core en otros proyectos académicos permite concentrar el tiempo en la calidad de la API y no en aprender un entorno nuevo dentro del plazo de la actividad.
4. Calidad técnica. ASP.NET Core integra validación, autenticación JWT, inyección de dependencias y documentación Swagger.

Se mantiene: el modelo de cuatro entidades, los roles Admin y Cliente, la regla central (la descarga en alta resolución solo con pago aprobado), el almacenamiento privado de archivos y el repositorio en GitHub. Cambia: la capa de presentación, pues el MVC con vistas Razor evoluciona hacia una API REST que devuelve JSON, y el flujo «crear reserva con comprobante» se divide en dos peticiones: crear la reserva y luego subir el comprobante. También cambia el gestor de base de datos, de MySQL a PostgreSQL.

1.Estructura del proyecto backend

Para esta segunda fase de VisualSync, y para cumplir el requerimiento de construir una API REST, la arquitectura planificada originalmente (MVC clásico con vistas Razor) evolucionó hacia una arquitectura en capas. Esto permite separar el backend de un futuro frontend independiente, comunicándose mediante peticiones HTTP y respuestas en formato JSON.

```
VisualSync.Api/

Controllers/

Models/

DTOs/

Services/

Data/

Migrations/
```

```
Middleware/

Storage/

appsettings.json

Program.cs
```

2. Implementación con C# y ASP.NET Core

Equivalente a la implementación con JavaScript/TypeScript sugerida en la guía, según el cambio autorizado.

Entorno: .NET SDK 10.0.401 , Visual Studio Code con la extensión C# Dev Kit, Postman y un proyecto de Supabase.

```
dotnet new webapi -n VisualSync.Api --use-controllers

cd VisualSync.Api

dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL

dotnet add package Microsoft.EntityFrameworkCore.Design

dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer

dotnet add package BCrypt.Net-Next

dotnet add package Swashbuckle.AspNetCore

dotnet add package SixLabors.ImageSharp

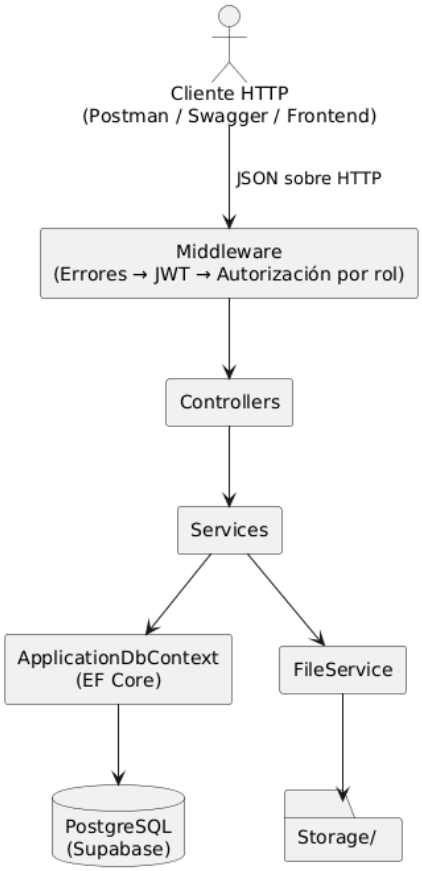
dotnet tool install --global dotnet-ef
```

Paquete	**Uso**

Npgsql.EntityFrameworkCore.PostgreSQL	Proveedor de Entity Framework Core para PostgreSQL
Microsoft.EntityFrameworkCore.Design	Herramientas de migraciones
Microsoft.AspNetCore.Authentication.JwtBearer	Autenticación con tokens JWT
BCrypt.Net-Next	Hash de contraseñas
Swashbuckle.AspNetCore	Documentación Swagger/OpenAPI
SixLabors.ImageSharp	Vistas previas en baja resolución

3. Arquitectura del backend

El backend sigue una arquitectura en capas con una API REST sin estado: cada petición lleva su token JWT y la API no guarda sesión.



Capa	**Responsabilidad**	**Regla**
Middleware	Capturar excepciones, validar token y rol	Todo error sale como JSON
Controllers	Recibir la petición, validar el DTO y devolver el código HTTP	Sin lógica de negocio
Services	Reglas de negocio: fechas, estados de pago, propiedad del recurso	No conocen HTTP: lanzan AppException
Data (EF Core)	Acceso a PostgreSQL con consultas parametrizadas	No se escribe SQL a mano

Storage	Archivos físicos fuera de la carpeta pública	Solo se entregan por un endpoint que verifica permisos

Decisiones de seguridad

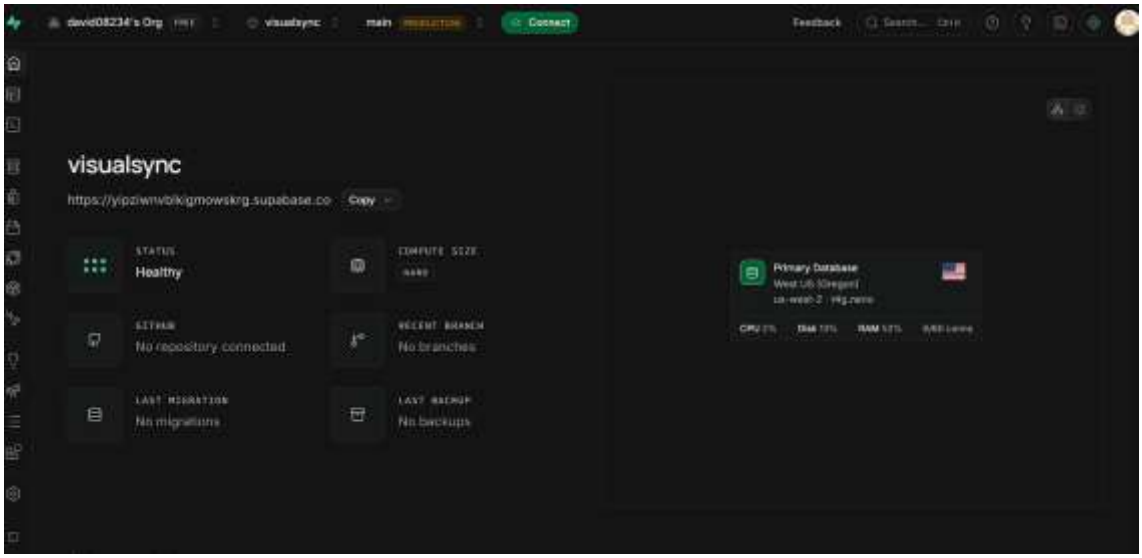
- Las contraseñas se guardan como hash con BCrypt.
- El registro público siempre crea usuarios con rol Cliente. El administrador inicial se crea al arrancar con credenciales tomadas de los secretos.
- El token contiene el identificador, el correo y el rol del usuario, y expira a las 2 horas.
- Un cliente solo accede a sus propias reservas y archivos.
- La descarga de la foto original exige pago aprobado y se verifica en el servidor.

Evolución respecto a la Actividad 1. Los controladores dejan de devolver vistas Razor, las rutas pasan a forma REST (/api/...) y las acciones de administración (/Admin/...) se convierten en endpoints protegidos por rol.

4. Configuración de PostgreSQL

Se utiliza Supabase, un servicio de PostgreSQL administrado en la nube, que evita instalar el gestor de forma local. Se creó un proyecto llamado visualsync en la región más cercana, se definió la contraseña de la base de datos y se obtuvo la cadena de conexión del Session pooler (puerto 5432), que admite tanto las migraciones como la aplicación. La cadena se guarda en User Secrets y no en el repositorio.

Supabase se usa únicamente como base de datos: la autenticación (JWT propio) y el almacenamiento de archivos se resuelven en la API.



5. Modelo y tablas de la base de datos

Se mantienen las cuatro entidades de la Actividad 1. Las enumeraciones se guardan como texto legible (por ejemplo, Aprobado).

Usuarios

Campo	**Tipo en PostgreSQL**	**Restricciones**
IdUsuario	integer (identidad)	PK
Rol	varchar(20)	Obligatorio (Admin o Cliente)
NombreCompleto	varchar(120)	Obligatorio
Email	varchar(150)	Obligatorio, único
PasswordHash	varchar(100)	Obligatorio

PaquetesServicio

Campo	**Tipo en PostgreSQL**	**Restricciones**

IdPaquete	integer (identidad)	PK
Nombre	varchar(100)	Obligatorio
Descripcion	varchar(500)	Obligatorio
Precio	numeric(10,2)	Obligatorio, mayor a 0
Duracion	integer	Obligatorio (horas)
CantidadFotos	integer	Obligatorio
IncluyeVideo	boolean	Obligatorio
Activo	boolean	Por defecto verdadero

Reservas

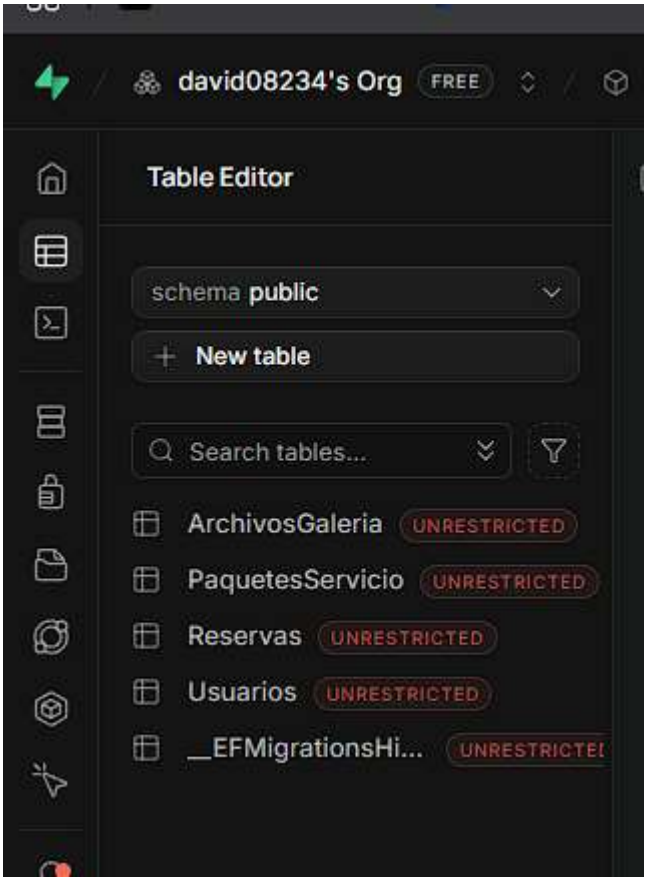
Campo	**Tipo en PostgreSQL**	**Restricciones**
IdReserva	integer (identidad)	PK
IdUsuario	integer	FK → Usuarios
IdPaquete	integer	FK → PaquetesServicio
FechaEvento	timestamp	Obligatorio
LugarEvento	varchar(200)	Obligatorio

EstadoReserva	varchar(30)	Solicitada, Confirmada, Finalizada o Cancelada
EstadoPago	varchar(30)	SinComprobante, PendienteValidacion, Aprobado o Rechazado
RutaComprobanteQR	varchar(300)	Opcional
MotivoRechazo	varchar(300)	Opcional
DescargasHabilitadas	boolean	Por defecto falso

ArchivosGaleria

Campo	**Tipo en PostgreSQL**	**Restricciones**
IdArchivo	integer (identidad)	PK
IdReserva	integer	FK → Reservas, eliminación en cascada
RutaBajaRes	varchar(300)	Obligatorio
RutaAltaRes	varchar(300)	Obligatorio
TipoArchivo	varchar(10)	Obligatorio (.jpg o .png)
SeleccionadaPorCliente	boolean	Por defecto falso

Relaciones: Usuario–Reserva (1:N), PaqueteServicio-Reserva (1:N) y Reserva-ArchivoGaleria (1:0..N). Las dos primeras usan eliminación restringida: no se puede borrar un usuario o un paquete que tenga reservas.



6. Conexión mediante ORM

Se utiliza Entity Framework Core con el proveedor Npgsql. El contexto ApplicationDbContext expone las tablas como DbSet y define las restricciones del modelo. La fecha del evento se mapea a timestamp sin zona horaria, para guardar la hora local tal como llega.

```
USING MICROSOFT.ENTITYFRAMEWORKCORE;
USING VISUALSYNC.API.MODELS;

NAMESPACE VISUALSYNC.API.DATA;

PUBLIC CLASS APPLICATIONDBCONTEXT : DBCONTEXT
{
    PUBLIC APPLICATIONDBCONTEXT(DBCONTEXTOPTIONS<APPLICATIONDBCONTEXT>
OPTIONS) : BASE(OPTIONS) { }

    PUBLIC DBSET<USUARIO> USUARIOS => SET<USUARIO>();
```

```
    PUBLIC DBSET<PAQUETESERVICIO> PAQUETESSERVICIO =>
SET<PAQUETESERVICIO>();
    PUBLIC DBSET<RESERVA> RESERVAS => SET<RESERVA>();
    PUBLIC DBSET<ARCHIVOGALERIA> ARCHIVOSGALERIA =>
SET<ARCHIVOGALERIA>();

    PROTECTED OVERRIDE VOID ONMODELCREATING(MODELBUILDER B)
    {
        BASE.ONMODELCREATING(B);

        B.ENTITY<USUARIO>().HASINDEX(U => U.EMAIL).ISUNIQUE();

        B.ENTITY<PAQUETESERVICIO>().PROPERTY(P =>
P.PRECIO).HASPRECISION(10, 2);

        B.ENTITY<RESERVA>().PROPERTY(R =>
R.FECHAEVENTO).HASCOLUMNTYPE("TIMESTAMP WITHOUT TIME ZONE");

        B.ENTITY<RESERVA>()
            .HASONE(R => R.USUARIO)
            .WITHMANY(U => U.RESERVAS)
            .HASFOREIGNKEY(R => R.IDUSUARIO)
            .ONDELETE(DELETEBEHAVIOR.RESTRICT);

        B.ENTITY<RESERVA>()
            .HASONE(R => R.PAQUETE)
            .WITHMANY(P => P.RESERVAS)
            .HASFOREIGNKEY(R => R.IDPAQUETE)
            .ONDELETE(DELETEBEHAVIOR.RESTRICT);

        B.ENTITY<ARCHIVOGALERIA>()
            .HASONE(A => A.RESERVA)
            .WITHMANY(R => R.ARCHIVOS)
            .HASFOREIGNKEY(A => A.IDRESERVA)
            .ONDELETE(DELETEBEHAVIOR.CASCADE);
    }
}
```

Registro en Program.cs:

```
BUILDER.SERVICES.ADDDBCONTEXT<APPLICATIONDBCONTEXT>(OPTIONS =>
    OPTIONS.USENPGSQL(BUILDER.CONFIGURATION.GETCONNECTIONSTRING("DEFAUL
T")));
```

El acceso a datos se hace con LINQ, que EF Core traduce a SQL parametrizado, lo que reduce el riesgo de inyección SQL: await _db.PaquetesServicio.Where(p => p.Activo).ToListAsync();

7. Migraciones

Las migraciones se generan con el enfoque Code-First a partir de las clases del modelo.

dotnet ef migrations add Inicial

dotnet ef database update

Paso	**Resultado**
migrations add Inicial	Crea la carpeta Migrations/ con el código que construye las cuatro tablas
database update	Ejecuta la migración en PostgreSQL y crea la tabla EFMigrationsHistory

Cada cambio posterior del modelo se registra con una nueva migración, lo que deja un historial versionado del esquema que se sube al repositorio.

8. Operaciones CRUD

El CRUD completo se implementa sobre Paquetes y se complementa con las operaciones de Reservas.

Operación	**Paquetes**	**Reservas**
Crear	POST /api/paquetes (Admin)	POST /api/reservas (Cliente)
Leer	GET /api/paquetes, GET /api/paquetes/{id}	GET /api/reservas, GET /api/reservas/{id}
Actualizar	PUT /api/paquetes/{id} (Admin)	PUT /api/reservas/{id} (dueño, solo en estado Solicitada)

Eliminar	DELETE /api/paquetes/{id} (baja lógica: Activo = false)	DELETE /api/reservas/{id} (cancelación: estado Cancelada)

Ejemplo de controlador (Paquetes)

```
[ApiController]
[Route("api/[controller]")]
public class PaquetesController : ControllerBase
{
    private readonly PaqueteService _service;
    public PaquetesController(PaqueteService service) => _service =
service;

    [HttpGet]
    public async Task<ActionResult<IEnumerable<PaqueteDto>>> Listar()
=>
        Ok(await _service.ListarActivosAsync());

    [HttpGet("{id:int}")]
    public async Task<ActionResult<PaqueteDto>> Obtener(int id) =>
        Ok(await _service.ObtenerAsync(id));

    [Authorize(Roles = "Admin")]
    [HttpPost]
    public async Task<ActionResult<PaqueteDto>> Crear(PaqueteCrearDto
dto)
    {
        var creado = await _service.CrearAsync(dto);
        return CreatedAtAction(nameof(Obtener), new { id =
creado.IdPaquete }, creado);
    }

    [Authorize(Roles = "Admin")]
    [HttpPut("{id:int}")]
    public async Task<ActionResult<PaqueteDto>> Actualizar(int id,
PaqueteCrearDto dto) =>
        Ok(await _service.ActualizarAsync(id, dto));

    [Authorize(Roles = "Admin")]
    [HttpDelete("{id:int}")]
    public async Task<IActionResult> Desactivar(int id)
    {
        await _service.DesactivarAsync(id);
        return NoContent();
    }
}
```

Reglas de negocio de las reservas

1. Al crear, el paquete debe existir y estar activo, la fecha debe ser futura y no puede haber otra reserva Confirmada ese día (409). La reserva nace como Solicitada con pago SinComprobante.
2. El comprobante solo se sube si el pago está en SinComprobante o Rechazado. Tras subirlo, el pago pasa a PendienteValidacion.
3. El administrador decide con PATCH /api/reservas/{id}/pago. Al aprobar, el pago pasa a Aprobado, la reserva a Confirmada y se habilitan las descargas, previa verificación de que la fecha siga libre (409). Al rechazar, es obligatorio indicar un motivo.
4. Un cliente solo ve, edita y cancela sus propias reservas (403 si intenta acceder a otra).

9. Validación de datos

Validación de entrada con Data Annotations en los DTOs. El atributo [ApiController] valida el DTO automáticamente y responde 400 si es inválido, sin llegar al servicio.

```
public class PaqueteCrearDto
{
    [Required(ErrorMessage = "El nombre es obligatorio.")]
    [StringLength(100)]
    public string Nombre { get; set; } = "";

    [Required, StringLength(500)]
    public string Descripcion { get; set; } = "";

    [Range(0.01, 1000000, ErrorMessage = "El precio debe ser mayor a
0.")]
    public decimal Precio { get; set; }

    [Range(1, 24)]
    public int Duracion { get; set; }

    [Range(1, 5000)]
    public int CantidadFotos { get; set; }
```

```
    public bool IncluyeVideo { get; set; }
}
```

El registro valida además el formato del correo ([EmailAddress]) y una contraseña de mínimo 8 caracteres ([MinLength(8)]).

Validación de reglas de negocio en los servicios. Fecha futura, fecha no ocupada, estado de pago válido para cada operación y propiedad del recurso. Si no se cumplen, el servicio lanza una AppException con el código HTTP correspondiente (400, 403, 404 o 409).

Validación de archivos. Antes de guardarlos se verifica el tamaño (comprobante: máximo 5 MB; fotografías: máximo 25 MB cada una), la extensión (.jpg, .jpeg o .png) y el tipo real mediante la firma binaria del archivo (JPEG: FF D8 FF; PNG: 89 50 4E 47), para que un archivo renombrado no pase el control. Los archivos se guardan con un nombre generado (GUID), no con el original.

10. Manejo de errores

Un middleware global de excepciones captura todos los errores y responde siempre con un JSON uniforme en formato ProblemDetails. Los servicios lanzan AppException con un código HTTP y el middleware lo traduce. Cualquier error inesperado se registra en el log y devuelve un 500 con un mensaje genérico, sin exponer detalles internos.

```
public class AppException : Exception
{
    public int StatusCode { get; }
    public AppException(int statusCode, string message) : base(message)
=> StatusCode = statusCode;
}

public class ExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionMiddleware> _logger;
    public ExceptionMiddleware(RequestDelegate next,
ILogger<ExceptionMiddleware> logger)
    { _next = next; _logger = logger; }

    public async Task InvokeAsync(HttpContext ctx)
```

```
    {
        try { await _next(ctx); }
        catch (AppException ex) { await Escribir(ctx, ex.StatusCode,
ex.Message); }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error no controlado");
            await Escribir(ctx, 500, "Ocurrió un error interno en el
servidor.");
        }
    }

    private static Task Escribir(HttpContext ctx, int status, string
detalle)
    {
        ctx.Response.StatusCode = status;
        return ctx.Response.WriteAsJsonAsync(new ProblemDetails
        {
            Status = status,
            Title = ReasonPhrases.GetReasonPhrase(status),
            Detail = detalle
        });
    }
}
```

Situación	**Código**	**Origen**
DTO inválido	400	Validación automática de [ApiController]
Regla de negocio incumplida	400 o 409	AppException en el servicio
Sin token o token inválido	401	Middleware de autenticación
Rol insuficiente o recurso ajeno	403	Autorización o servicio
Recurso inexistente	404	AppException en el servicio

Error inesperado	500	Middleware global

11. Endpoints REST

Todos los endpoints cuelgan de /api y trabajan con JSON, salvo los de subida de archivos (multipart/form-data) y los de descarga.

#	**Método**	**Endpoint**	**Acceso**	**Descripción**	**Respuestas**
1	POST	/api/auth/registro	Público	Crea una cuenta de Cliente	201, 400, 409
2	POST	/api/auth/login	Público	Devuelve el token JWT	200, 400, 401
3	GET	/api/paquetes	Público	Lista los paquetes activos	200
4	GET	/api/paquetes/{id}	Público	Detalle de un paquete	200, 404
5	POST	/api/paquetes	Admin	Crea un paquete	201, 400, 401, 403
6	PUT	/api/paquetes/{id}	Admin	Actualiza un paquete	200, 400, 401, 403, 404

7	DELETE	/api/paquetes/{id}	Admin	Desactiva un paquete	204, 401, 403, 404
8	GET	/api/reservas/fechas-ocupadas	Autenticado	Fechas con reserva confirmada	200, 401
9	GET	/api/reservas	Autenticado	Cliente: las suyas. Admin: todas	200, 401
10	GET	/api/reservas/{id}	Dueño o Admin	Detalle de una reserva	200, 401, 403, 404
11	POST	/api/reservas	Cliente	Crea una reserva (Solicitada)	201, 400, 401, 404, 409
12	PUT	/api/reservas/{id}	Dueño	Edita fecha y lugar (solo Solicitada)	200, 400, 401, 403, 404, 409
13	DELETE	/api/reservas/{id}	Dueño o Admin	Cancela la reserva	204, 401, 403, 404, 409

14	POST	/api/reservas/{id}/comprobante	Dueño	Sube el comprobante (multipart)	200, 400, 401, 403, 404, 409
15	PATCH	/api/reservas/{id}/pago	Admin	Aprueba o rechaza el pago	200, 400, 401, 403, 404, 409
16	POST	/api/reservas/{id}/archivos	Admin	Sube un lote de fotografías (multipart)	201, 400, 401, 403, 404
17	GET	/api/reservas/{id}/archivos	Dueño o Admin	Lista los archivos de la galería	200, 401, 403, 404
18	GET	/api/archivos/{id}/miniatura	Dueño o Admin	Vista previa en baja resolución	200, 401, 403, 404
19	GET	/api/archivos/{id}/descargar	Dueño con pago aprobado, o Admin	Foto original en alta resolución	200, 401, 403, 404

12. JSON y códigos HTTP

Registro: POST /api/auth/registro

```
1 { "nombreCompleto": "Lucía Martín", "email": "lucia@correo.com",
  "password": "Clave12345" }
```

Respuesta 201 Created:

```
1 { "idUsuario": 2, "nombreCompleto": "Lucía Martín", "email":
  "lucia@correo.com", "rol": "Cliente" }
```

Login: POST /api/auth/login. Respuesta 200 OK:

```
1 { "token": "eyJhbGciOiJIUzI1NiIs...", "expiraEnMinutos": 120,
2   "usuario": { "idUsuario": 2, "nombreCompleto": "Lucía Martín", "rol":
  "Cliente" } }
```

Crear reserva: POST /api/reservas (cabecera Authorization: Bearer token)

```
1 { "idPaquete": 1, "fechaEvento": "2026-11-15T16:00:00", "lugarEvento":
  "Salón Los Pinos, Cochabamba" }
```

Respuesta 201 Created:

```
1 { "idReserva": 5, "idPaquete": 1, "fechaEvento": "2026-11-15T16:00:00",
2   "lugarEvento": "Salón Los Pinos, Cochabamba", "estadoReserva":
  "Solicitada",
3   "estadoPago": "SinComprobante", "descargasHabilitadas": false }
```

Aprobar o rechazar pago: PATCH /api/reservas/{id}/pago (Admin). Para aprobar: { "aprobar": true }. Para rechazar: { "aprobar": false, "motivo": "El monto no coincide con el paquete." }.

Error de validación (400):

```
1  { "title": "One or more validation errors occurred.", "status": 400,
2    "errors": { "Precio": ["El precio debe ser mayor a 0."] } }
```

Error de negocio (409):

```
1  { "title": "Conflict", "status": 409,
2    "detail": "La fecha seleccionada ya está ocupada por una reserva confirmada." }
```

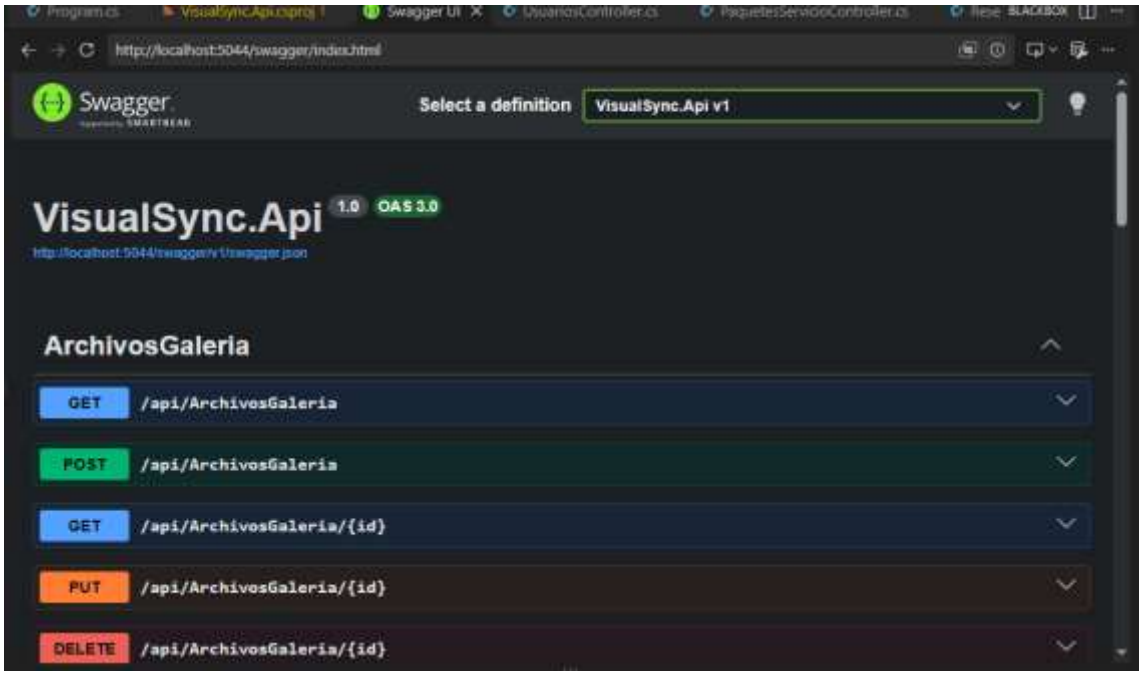
Códigos HTTP utilizados

Código	**Significado**	**Uso en VisualSync**
200 OK	Éxito	Consultas, actualizaciones, login, subida del comprobante
201 Created	Recurso creado	Registro, paquetes, reservas, archivos
204 No Content	Éxito sin cuerpo	Desactivar paquete, cancelar reserva
400 Bad Request	Datos inválidos	Validación de DTOs y de archivos
401 Unauthorized	Sin autenticación	Token ausente o vencido, credenciales incorrectas

403 Forbidden	Sin permiso	Rol insuficiente, recurso ajeno, pago no aprobado
404 Not Found	No existe	Paquete, reserva o archivo inexistente
409 Conflict	Conflicto de estado	Fecha ocupada, correo repetido, pago en estado no válido
500 Internal Server Error	Error del servidor	Excepción no controlada

13. Documentación básica de API

La API se documenta con Swagger/OpenAPI (Swashbuckle), disponible en /swagger. Lista los 19 endpoints con sus parámetros, cuerpos de ejemplo y respuestas, y permite probarlos desde el navegador. Con el botón Authorize se ingresa el token JWT (Bearer token) para los endpoints protegidos. Además, el repositorio incluye una colección de Postman (docs/VisualSync.postman_collection.json) con las pruebas de cada endpoint.



Pruebas de funcionamiento

#	**Prueba**	**Endpoint**	**Resultado esperado**	**Resultado obtenido**
1	Registrar cliente	POST /api/auth/registro	201	Exitoso
2	Registro con correo repetido	POST /api/auth/registro	409	Exitoso
3	Login correcto	POST /api/auth/login	200 con token	Exitoso
4	Login con clave incorrecta	POST /api/auth/login	401	Exitoso
5	Crear paquete como Admin	POST /api/paquetes	201	Exitoso
6	Crear paquete como Cliente	POST /api/paquetes	403	Exitoso
7	Crear paquete con precio inválido	POST /api/paquetes	400	Exitoso

8	Listar, editar y desactivar paquete	GET, PUT, DELETE	200, 200, 204	Exitoso
9	Crear reserva	POST /api/reservas	201	Exitoso
10	Subir comprobante	POST /api/reservas/{id}/comprobante	200	Exitoso
11	Subir archivo con formato inválido	POST /api/reservas/{id}/comprobante	400	Exitoso
12	Aprobar pago	PATCH /api/reservas/{id}/pago	200	Exitoso
13	Descargar con pago pendiente	GET /api/archivos/{id}/descargar	403	Exitoso
14	Descargar con pago aprobado	GET /api/archivos/{id}/descargar	200 con archivo	Exitoso
15	Ver la reserva de otro cliente	GET /api/reservas/{id}	403	Exitoso

14. Repositorio actualizado

- Plataforma: GitHub.
- Repositorio: https://github.com/david08234/VisualSyncMVC

- Contenido: el proyecto VisualSync.Api/ con el backend completo (incluida la carpeta Migrations/), la carpeta docs/ con el documento de la Actividad 1, los diagramas y la colección de Postman, y un README.md con los requisitos y las instrucciones para ejecutar el proyecto.
- Ramas: main (estable) y ramas feature/... por funcionalidad, que se fusionan en main.
- Commits: mensajes descriptivos asociados a las historias de usuario, por ejemplo feat(HU-07): agregar registro y login con JWT, feat(HU-04): aprobar y rechazar pagos y fix(HU-03): validar tipo real del comprobante.
- Archivos ignorados: bin/, obj/, .vs/, appsettings.Development.json y Storage/, que contiene comprobantes y fotografías de clientes. Las credenciales se gestionan con User Secrets y no se publican.

